

FlyAda: Belief-State Adaptation for Diffusion Policies under Partial Observation

Lingxue Lyu*

University of Pennsylvania
Philadelphia, Pennsylvania, United States
lingxuelyu@alumni.upenn.edu

Abstract—We study when a small online-updated latent helps an action-chunk diffusion policy on a UAV goal-reaching task with mass, drag, wind, and control-delay perturbations. With the 12-dim state and the goal-relative vector visible to the policy, vanilla diffusion already does well on the conventional mass/drag/wind/delay sweep, including extrapolation conditions ($4\times$ mass, $10\times$ drag, 8-step delay). We attribute this to receding-horizon re-sampling at $K=4$: the policy replans from the current (perturbed) state often enough that the next chunk corrects most of the dynamics error. Adding an adaptation latent on top of the vanilla policy gained essentially nothing on the full-observation sweep, and on the most extreme extrapolation cells it sometimes hurt. The original problem reappears under partial observation. With velocity zeroed in the policy’s input at training and at test, the same diffusion policy collapses on the same sweep (0.3% mean success), and a 3-frame-stack baseline does almost as badly (1.1%). FlyAda, the configuration that closes the gap, adds a small observer head trained with an auxiliary velocity-decoding loss — supervised with the true velocity, a privileged training-time signal that is itself never observed at deployment; the latent is updated online at deployment with an EMA. FlyAda hits near-full success on the partial-obs sweep within the training perturbation range and degrades only at wind well above training (e.g. 0.95 at 2 m/s, 0 at 3 m/s). A same-weights ablation that toggles only the test-time update rule points at the continuous online update as the part that matters. A linear probe shows z_t recovers the hidden velocity to $R^2 = 0.992$. The same gap shows up when we re-evaluate the checkpoints on a 6-DoF MuJoCo quadrotor.

Keywords— Diffusion policies, UAV control, receding-horizon, partial observability, learned observer, test-time adaptation.

I. INTRODUCTION

Action-chunk diffusion policies [1] are a strong imitation-learning recipe for continuous control. A common assumption is that they should execute poorly when test-time dynamics shift away from training, since the denoiser conditions on a single observation. Our prior work on language-conditioned UAV control [2] found that a small online-updated latent helped close a similar mismatch gap, and we initially intended to port that recipe to the diffusion setting. The first set of experiments did not show the gap we were trying to close. Under full observation, vanilla diffusion was already at near 100% across the conventional sweep, and the latent didn’t add anything on top.

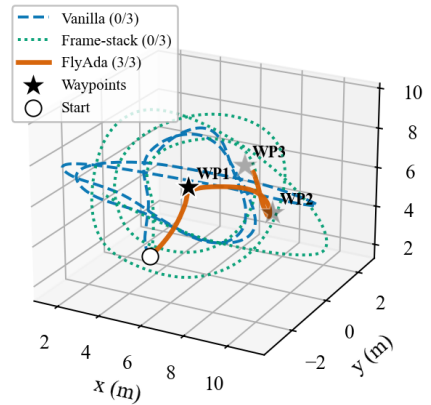


Fig. 1. Same partial-obs 3-waypoint task, three policies, one seed. With velocity hidden, vanilla (blue, dashed) and a 3-frame stack (green, dotted) drift past the first waypoint and never recover. FlyAda (orange) adds a small observer head trained with a velocity-decoding auxiliary loss and threads all three.

We had to restrict the policy’s input to recover a non-trivial gap. With velocity zeroed in the observation, the same diffusion policy collapses on the same sweep. A 3-frame stack didn’t help; we attribute this to the diffusion MSE objective providing no direct gradient that asks the model to extract velocity from position deltas. What did help was an auxiliary loss that decodes the hidden velocity from the latent.

FlyAda combines three ingredients — an online-updated latent in the spirit of our prior work [2], an EMA update structurally similar to the online calibration update used in autonomous-vehicle longitudinal control [3], and an auxiliary velocity-decoding head on the latent — and all three have to be present for the partial-observation gap to close on the UAV goal-reach task we use.¹ Frame-stacking does not close it, and a linear probe on the trained latent recovers the hidden velocity to $R^2 = 0.992$ rather than encoding perturbation identity. Whether the same recipe scales to higher-DoF dynamics or to less-observable parameters than velocity is an open question we begin to address in §X. Every

¹An earlier version of the auxiliary head decoded log-mass instead of velocity. That head trained to $R^2 \approx 0.86$ on the demo distribution, which seemed promising; but the deployed policy was no better than vanilla, and a same-weights ablation showed the latent simply wasn’t being read by the denoiser in a way that affected behaviour. That dead end is what pointed us at the velocity-observability angle.

result here is simulation-only; §XI gives the real-hardware caveats.

II. RELATED WORK

Diffusion policies. Action-chunk diffusion [1] with DDPM [4], [5] training and DDIM [6] sampling is the backbone we use. Most prior evaluations focus on in-distribution behaviour; our full-observation result is one data point on robustness under dynamics mismatch.

Test-time adaptation and learned state observation. RMA [7] trains a privileged-information policy first and then an inference module that maps observation history to the privileged latent. Test-time training [8] and self-supervised policy adaptation [9] update model parameters online. Closer to our setting, our own AeroBridge-TTA [2] uses an online-updated latent for language-conditioned UAV control and reports a $4.6\times$ OOD lift from continuous test-time updating with the same weights. FlyAda is in the same family but uses a supervised velocity-decoding loss on a small MLP latent rather than a separate inference module, and it targets the diffusion-policy regime.

Adaptive flight and residual dynamics. Neural-Fly [10] and Neural Lander [11] learn residual dynamics for agile quadrotor flight under wind and ground effect. Classical adaptive control [12] formalises the same idea with Lyapunov stability arguments. Our setting is the imitation-learning regime where the controller is a denoiser, not a structured model-based controller.

Robust execution under partial observability. Privileged-information teacher policies distilled to partial-observation students are a standard sim-to-real recipe in legged locomotion [13], [14]; the auxiliary loss in Sec. VI is in the same family. Recurrent policies are the alternative when the missing channel is harder to identify a priori, as in dexterous in-hand manipulation [15]; on broader POMDP benchmarks, model-free RL with a recurrent encoder is reported as a strong baseline [16]. From the control side, multi-contact fall-mitigation planning for humanoids [17], [18], [19] and self-balancing bicycle control [20] both argue that robust execution needs an adaptation step inside the controller. Closest to our actual update rule, online longitudinal calibration for autonomous vehicles [3] uses an EMA-residual update structurally identical to ours on $\mathbf{z}_t$. Domain randomisation [21], [22] is complementary; for the UAV setting here the diffusion-policy frontier appears to be set by observation structure rather than training-distribution width.

III. SETUP

State and action. The internal state is $\mathbf{x}_t = [\mathbf{p}, \mathbf{v}, \text{yaw}, \dot{\text{yaw}}, \mathbf{g}_{\text{rel}}, d_{\text{goal}}] \in \mathbb{R}^{12}$. Action $\mathbf{u}_t \in [-1, 1]^3$ is a normalised acceleration command scaled by 5 m/s^2 per axis. $\Delta t = 0.02 \text{ s}$, 400-step horizon on Task A.

Dynamics and perturbations. $\mathbf{v}_{t+1} = \mathbf{v}_t + \Delta t(\mathbf{u}_t^{\text{applied}}/m - c_d \mathbf{v}_t + \mathbf{w}_t)$, nominal $m=1$, $c_d=0.1$, positions integrated semi-implicitly ($\mathbf{p}_{t+1} = \mathbf{p}_t + \Delta t \mathbf{v}_{t+1}$). $\mathbf{u}_t^{\text{applied}} = \mathbf{u}_{t-\tau}$ is popped from a FIFO delay queue on

TABLE I
FULL-OBS SWEEP SUCCESS. PLAN-SPECIFIED ROW AVERAGES OVER 12 CONDITIONS (30 SEEDS EACH); EXTRAPOLATION ROWS ARE AT 20 SEEDS.

Sweep regime	Vanilla	FlyAda
Plan-specified (12 conds)	1.000	0.964
mass $4\times$ (extrap.)	0.95	0.50
drag $10\times$ (extrap.)	0.95	0.95
wind 3 m/s (extrap.)	0.75	0.55
delay 8 steps (extrap.)	0.95	0.65

the commanded action (zero-filled at reset), τ the swept delay-steps parameter. Yaw/ $\dot{\text{yaw}}$ fill out a quadrotor-shaped state but are held at 0 here; multi-axis attitude/rotor dynamics enter only through the 6-DoF MuJoCo inner loop (Sec. X). The training-range sweep uses $m \in [1.0, 1.3]$, $c_d \in [0.1, 0.2]$, wind up to 1.5 m/s, delay up to 3 env steps. Extrapolation goes well beyond.

Tasks. Task A is goal-reaching: spawn in a $\pm 5 \text{ m}$ box, goal 3–7 m away, 0–2 box/cylinder obstacles, success at $d_{\text{goal}} < 0.5 \text{ m}$ and $\|\mathbf{v}\| < 1 \text{ m/s}$ within 400 steps. Task B chains 3 sequential waypoints (each 2.5–4.5 m apart); $\mathbf{g}_{\text{rel}}/d_{\text{goal}}$ in the obs always point to the current waypoint and advance when it is reached, with an 800-step budget.

Observation modes. *Full*: the policy sees the 12-dim state. *Partial*: velocity dims are zeroed in the observation; the environment integrates with the true velocity internally. The PPO teacher and demo collection always use full observation.

Demos. A Stable-Baselines3 [23] PPO [24] MLP teacher (3 M env steps, 100% on a 60-seed deterministic eval) provides 50 067 nominal transitions and 50 023 transitions under perturbations sampled from the training-range sweep. Diffusion-based policies in this paper share this pool.

Metrics. Success rates are Bernoulli proportions over n seeds (per caption); 95% Wilson half-widths are $\leq 0.09/0.06/0.04$ for $n=20/30/40$ at the near-0/near-1 rates dominating our tables, widening to ± 0.15 – 0.20 only near $p=0.5$ (flagged as noise in-text, e.g. Sec. IX).

IV. DIFFUSION HOLDS UP UNDER FULL OBSERVATION

We trained a vanilla diffusion policy (1D temporal UNet, 3.98 M parameters, FiLM conditioning on timestep + state embedding) for 30 epochs on the nominal demos and evaluated under the conventional sweep. Vanilla success was at 1.0 on every cell of the plan-specified sweep (30 seeds each).

Pushed an order of magnitude past the plan-specified range, vanilla was still at ~ 0.95 on mass $4\times$, drag $10\times$, and delay 8. Adding the adaptation latent on top didn't gain anything here, and on the most extreme extrapolation it sometimes hurt because the latent went off-distribution.

A few things probably contribute. The policy re-plans every $K=4$ env steps from the current state, which lets the next chunk absorb most of the previous chunk's dynamics error before it has time to compound. $\mathbf{g}_{\text{rel}}$ and d_{goal} close the spatial loop geometrically — the goal-relative input already encodes whatever has drifted. The dynamics themselves are also low-dimensional. Whatever the relative weights of these

TABLE II

PARTIAL-OBS MISMATCH SWEEP SUCCESS RATES. TOP: WITHIN TRAINING RANGE ($n=30$). BOTTOM: EXTRAPOLATION ($n=20$). WIND IN M/S; DELAY IN ENV STEPS.

Axis / level	Vanilla	Frame-stack	FlyAda
<i>Within training range</i>			
nominal	0.03	0.00	1.00
mass +30%	0.00	0.00	1.00
drag +50%	0.00	0.00	1.00
drag +100%	0.00	0.00	1.00
wind 1.0	0.00	0.00	1.00
wind 1.5	0.00	0.03	1.00
delay 3	0.00	0.00	1.00
<i>Beyond training range</i>			
mass +50%	0.00	0.00	1.00
mass +100%	0.00	0.05	1.00
drag +200%	0.00	0.00	1.00
wind 2.0	0.00	0.00	0.95
wind 3.0	0.00	0.00	0.00
delay 5	0.00	0.05	1.00

effects, the gap we set out to close is not the dominant failure mode under full observation, so we look elsewhere for it.

V. HIDING VELOCITY BREAKS THE CLOSED LOOP

POMDP setup. The Markov state $\mathbf{x}_t = [\mathbf{p}, \mathbf{v}, \text{yaw}, \dot{\text{yaw}}, \mathbf{g}_{\text{rel}}, d_{\text{goal}}]$ evolves under the dynamics in Sec. III, but the policy sees an observation $\mathbf{o}_t = H\mathbf{x}_t$ that drops the velocity component. The expert policy is Markovian on $\mathbf{x}_t$, so the imitation problem on $\mathbf{o}_t$ requires the policy (or some external module) to recover the missing state component. The full-generality POMDP solution would track a belief $b_t = p(\mathbf{x}_t | \mathbf{o}_{1:t}, \mathbf{u}_{1:t-1})$ [25]; in our deterministic dynamics the relevant part of that belief is just the missing channel itself, and we go after that channel directly. RNN-based recurrent policies [26] are another option in this slot; we discuss the trade-off in Sec. XI.

We then zeroed the three velocity dims in the observation, both at training (demos saved with velocity masked) and at test. The environment still integrates with the true velocity; only the policy’s view of the state loses it. We re-trained the same diffusion backbone for 30 epochs.

Nominal Task A: 0% success on 30 seeds, mean final distance 3.71 m (vs 0.50 m with full obs), almost all episodes ran to the 400-step truncation. Mean success across the 12-condition mismatch sweep was 0.3% (Table II, first column). The typical failure was overshoot — the policy accelerated toward the goal correctly, then couldn’t tell when to brake.

Frame stacking. The natural fix is more observation history. Position over the last K timesteps carries enough signal, in principle, for a finite-difference estimate of velocity. We trained a frame-stacking baseline with the same backbone, same demos, same training budget, but with a 36-dim conditioning vector $[\mathbf{s}_t-2, \mathbf{s}_t-1, \mathbf{s}_t]$ and zero-padding at the episode boundary.

Nominal success stayed at 0% after 30 epochs. The sweep mean was 1.1% (Table II, middle column); the small non-zero entries are one or two lucky rollouts out of twenty. The diffusion ϵ -MSE dropped normally during training ($0.14 \rightarrow 0.05$). We checked separately that the position differences

carry the right signal: a hand-coded finite-difference estimator recovers velocity to small RMSE at our 50 Hz rate. So the issue isn’t information access.

An imitation-ambiguity argument. Two history windows that share the most-recent position and lead to the same demo action chunk but correspond to different velocities are mapped to the same target by the diffusion MSE loss — same conditioning, same gradient. Frame-stacking enlarges the representable history space but does not change the supervisory signal, so the velocity-discriminating direction is never exercised. Demonstrations passing through similar position trajectories at different speeds are exactly such pairs, and they are common in our demo set. The auxiliary loss in Sec. VI attaches a per-step supervisory signal to a learned channel of the conditioning, which breaks the ambiguity directly.

VI. FLYADA: A SMALL OBSERVER WITH AN AUXILIARY LOSS

FlyAda keeps the single-frame observation and the same diffusion backbone. The only addition is a small observer head $f_\phi(\mathbf{s}_t, \mathbf{u}_t, \mathbf{s}_{t+1})$: a two-hidden-layer MLP (Mish, tanh output, 32 latent dims, about 73K params), with its final linear layer zero-initialised so $\mathbf{z}_t = \mathbf{0}$ at episode start. The latent is updated each environment step by

$$\mathbf{z}_{t+1} = (1 - \alpha)\mathbf{z}_t + \alpha f_\phi(\mathbf{s}_t, \mathbf{u}_t, \mathbf{s}_{t+1}), \quad (1)$$

with $\alpha=0.3$, and concatenated with the observation as conditioning to the diffusion denoiser.

EMA vs. additive accumulation. Equation 1 is an EMA. Additive accumulation ($\mathbf{z}_{t+1} = \mathbf{z}_t + \alpha f_\phi$) was our first attempt; $\|\mathbf{z}_t\|$ grew unboundedly over the 80-step training rollouts, the conditioning went off-distribution, and nominal success stalled around 7% even at 25 epochs. EMA bounded $\mathbf{z}_t$ to roughly f_ϕ ’s tanh output range and training proceeded. $\alpha=0.1$ and $\alpha=0.5$ also worked but less well; 0.3 was the value we kept. Not exhaustively swept.

Auxiliary velocity-decoding loss. The piece that actually closes the gap is supervision. We add a linear head $\text{dec}_\phi : \mathbb{R}^{32} \rightarrow \mathbb{R}^3$ and a loss

$$\mathcal{L}_{\text{vel}} = \mathbb{E}_t \|\text{dec}_\phi(\mathbf{z}_t) - \mathbf{v}_t^{\text{true}}\|^2, \quad (2)$$

with $\mathbf{v}_t^{\text{true}}$ the unmasked velocity from the demos. Total loss $\mathcal{L} = \mathcal{L}_{\text{diff}} + \lambda \mathcal{L}_{\text{vel}}$, $\lambda=5$. dec_ϕ is not used at deployment; it exists only as a gradient path that pushes $\mathbf{z}_t$ to encode velocity. Without this term ($\lambda=0$, otherwise identical config/budget), f_ϕ did not appear to converge to anything useful in our runs — nominal success reaches only 0.033 ($n=30$; Table III), and the loss curves looked almost identical to the frame-stack run.

Training behaviour. Figure 2 shows the diffusion loss and $\|\mathbf{z}_t\|$ over 25 epochs. The velocity-decoding loss (not plotted) drops from 1.94 to 0.045 over the same span. Nominal rollout success transitions abruptly from 0 to 93% once $\mathcal{L}_{\text{vel}}$ falls below ~ 0.17 at epoch 14, and locks in at 100% by epoch 19. The transition is sharp; we did not look further into why.

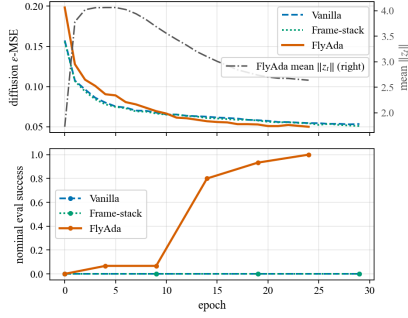


Fig. 2. Partial-obs training for the three methods. Top: diffusion ϵ -MSE; the three loss curves track each other within noise, so the training loss alone does not tell which method will succeed. Right axis: FlyAda’s mean $\|z_t\|$ (red, dash-dot). Bottom: nominal eval success rate during training. FlyAda jumps from 0 to 1 around epochs 14–19; vanilla and frame-stack stay at floor.

TABLE III

ABLATION ON THE HARDER COMBINED CONDITION (MASS $\times$ 1.5, DRAG $\times$ 4, WIND 2 M/S, DELAY 4), 40 SEEDS. SEPARATELY, A FROM-SCRATCH FLYADA TRAINED WITH $\lambda=0$ (NO $\mathcal{L}_{vel}$, SEC. VI), EVALUATED ON *nominal* ($n=30$, NOT THIS HARD CONDITION), REACHES ONLY 0.033 SUCCESS.

Test-time update rule	Succ.	d (m)
Vanilla (no z_t , separate weights)	0.000	2.91
Frame-stack (3-frame, separate weights)	0.000	3.06
FlyAda, $\alpha=0$ ($z_t \equiv 0$)	0.050	1.60
FlyAda, z_t frozen after 10 steps	0.025	3.61
FlyAda, continuous $\alpha=0.3$ EMA	0.100	0.68

Implementation details. The denoiser is a 1D temporal UNet (3.98 M params, hidden 128) with FiLM conditioning on a sinusoidal timestep embedding plus the (s_t, z_t) embedding; $T_{train}=50$, $T_{sample}=20$ (DDIM, cosine $\bar{\alpha}$); $H=8$, $K=4$. f_ϕ has hidden 128, latent 32, $\sim 73K$ params, EMA $\alpha=0.3$, aux weight $\lambda=5$. Optimizer AdamW, lr 10^{-4} , weight decay 10^{-6} . Vanilla and frame-stack train at batch 128 for 30 epochs (single RTX 5080, ~ 3 min/run); FlyAda trains at batch 32 for 25 epochs (~ 8 min) because each step rolls z_t along 80-step trajectories.

Reading Table II. FlyAda is at 1.0 across the entire training-range block and stays at 1.0 when extrapolating 50–100% on mass and 200% on drag. The condition that breaks it is wind: 0.95 at 2 m/s, 0 at 3 m/s with mean final distance 0.69 m. The drone gets close to the goal but can’t stop — the 5 m/s² horizontal acceleration cap is not enough to counter a 3 m/s sustained wind in our budget. Mean final distance stays under 0.50 m everywhere except wind ≥ 2 m/s. Action smoothness $\mathbb{E}[\|\Delta u_t\|^2]$ is 0.028, vs 0.020 for vanilla full-obs.

VII. ABLATION

To separate the contributions of network capacity and online latent updating, we freeze the FlyAda weights and toggle only the test-time update rule on a deliberately hard combined condition (mass $\times$ 1.5, drag $\times$ 4, wind 2 m/s, delay 4 steps; 40 seeds). The condition is severe enough that even FlyAda full doesn’t run away with it.

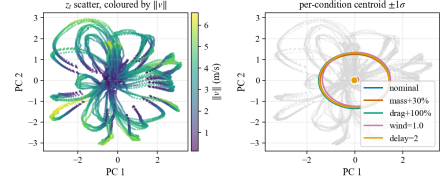


Fig. 3. Latent analysis. PCA of z_t , top two components (52.6% variance), $\sim 10\,500$ points from 5 dynamics conditions. **Left:** scatter coloured by $\|v\|$ — the radial direction tracks velocity magnitude. **Right:** per-condition centroid (dot) $\pm 1\text{-}\sigma$ covariance ellipse over the same projection. The five conditions superpose; a linear classifier $z_t \rightarrow \text{condition}$ gets only 24.2% accuracy (20% chance).

Table III is best read as a relative ordering. $\alpha=0$ pins the latent at 0 for the whole episode (capacity control); the frozen row identifies a z_t from the first 10 env steps and then freezes it (one-shot ID control). Continuous EMA updating beats both, with a $\sim 4\times$ success-rate gap and mean final distance pulled from 1.6–3.6 m to 0.68 m. Absolute success is low because every axis is at or past FlyAda’s training upper bound; this condition reappears as the Task B Extreme row.

VIII. WHAT DOES z_t ENCODE?

The ablation tells us continuous online updating matters but not what z_t encodes. Three probes.

Linear probe. FlyAda was rolled on 25 seeds each across nominal and four single-axis perturbations (mass+30%, drag+100%, wind 1, delay 2), recording (z_t, v_t^{true}) at every env step. On a 70/30 split of the $\sim 10K$ resulting pairs, an OLS regressor $z \mapsto v$ achieves $R^2 = 0.992$ on the held-out half (per axis: 0.99, 0.99, 0.99). z_t is, to a good approximation, an affine function of the true velocity — which is unsurprising given that dec_ϕ is a single linear layer and the joint training optimises for exactly that.

PCA structure. Figure 3 projects z_t to its top two PCA components (52.6% of variance: 29.3% + 23.3%). The left panel colours points by $\|v\|$: low-speed points cluster near the origin, high-speed points sit on the periphery, with the radial direction roughly tracking velocity magnitude. The right panel asks the dual question — does z_t carry the perturbation *type*? — by drawing each condition’s centroid and $1\text{-}\sigma$ covariance ellipse over the same projection. The five ellipses trace nearly the same curve and the centroids land on top of each other; conditions are essentially superposed in latent space. As a quantitative check, a linear classifier $z_t \rightarrow \text{condition}$ (5-way) reaches only 24.2% test accuracy, barely above the 20% chance level. So z_t specialises to its supervisory target (velocity) and gives up almost no extra capacity to identifying which dynamics regime it is in.

Smoothness. Mean $\|z_{t+1} - z_t\|$ is 0.12 overall and sits in $[0.10, 0.12]$ across all five conditions. The perturbed conditions don’t make z_t noisier; this is consistent with the EMA bound being the dominant regulariser.

IX. LONG-HORIZON TASK B

We re-evaluated the partial-observation checkpoints on Task B (3-waypoint chain, 800-step budget). No retraining: $g_{\text{rel}}/d_{\text{goal}}$ in the observation simply advance to the next

TABLE IV

TASK B (3-WAYPOINT, PARTIAL OBS, 30 SEEDS). COLS: TASK SUCCESS / WP FRACTION / EPISODE LENGTH / TRACKING ERROR MEAN, RMSE, MAX (M), ALL OVER THE FULL EPISODE. HARD = MASS+30%, DRAG+100%, WIND 1, DELAY 2. EXTREME = TABLE III CONDITION.

Cond.	Method	Succ.	WP	Len	Err (mn/RMSE/mx)
Nominal	Vanilla	0.000	0.11	800	3.00/3.17/5.05
	Frame-stack	0.000	0.07	794	2.74/2.88/4.57
	FlyAda	1.000	1.00	220	2.04/2.34/4.04
Hard	Vanilla	0.000	0.02	772	2.85/3.00/4.65
	Frame-stack	0.000	0.07	788	2.92/3.09/4.99
	FlyAda	0.933	0.96	307	2.02/2.32/4.07
Extreme	Vanilla	0.000	0.00	771	3.05/3.22/5.05
	Frame-stack	0.000	0.00	783	3.13/3.30/5.10
	FlyAda	0.000	0.12	790	1.07/1.31/3.76

TABLE V

MUJoCo 6-DOF TRANSFER ($n=20$, PARTIAL OBS). CELLS: SUCCESS / MEAN FINAL DISTANCE (M). HARD = MASS $\times$ 1.3, DRAG, WIND 1 M/S.

Condition	Vanilla	Frame-stack	FlyAda
Nominal	0.00 / 8.8	0.00 / 7.5	0.40 / 2.2
Hard	0.00 / 2.8	0.00 / 3.1	0.85 / 0.7

waypoint when the current one is reached, so a single-goal policy transfers by construction.

Transfer was uneventful: the observer keeps tracking velocity even when $\mathbf{g}_{\text{rel}}$ jumps discontinuously at waypoint advances, and the policy treats the next waypoint as a fresh goal. Vanilla and frame-stack plateau at $\sim 2\text{--}11\%$ waypoint completion in nominal and hard, consistent with “the drone drifts through the first waypoint by chance.” All three are at floor on strict success in Extreme, so WP-fraction and tracking error separate them instead: FlyAda reaches 0.12 WP fraction at 1.07/1.31/3.76 m mean/RMSE/max tracking error, versus ≤ 0.00 WP fraction and $\sim 3.0\text{--}3.3/5.0\text{--}5.1$ m for both baselines — FlyAda’s worst-case (max) deviation stays below the baselines’ typical (mean) deviation in every row.

X. SIM-TO-SIM TRANSFER TO A 6-DOF MUJoCo QUADROTOR

Our home simulator is a 3-DoF translational point-mass with linear drag. Real quadrotors are 6-DoF rigid bodies whose acceleration command goes through an attitude controller, rotor dynamics, and rigid-body rotation before producing translation. We do not have a real drone; what we did is re-evaluate the partial-obs checkpoints on a minimal MuJoCo [27] quadrotor: 1 kg, 0.15 m arm length, four rotors with the standard yaw-drag coupling, 500 Hz simulation, hidden velocity in the policy’s observation. An inner cascaded controller maps the policy’s 3-dim normalised acceleration into rotor commands by computing the required body-z thrust, deriving the desired body-z direction from the desired world-frame net force, and running an attitude-rate PID. The policy still issues actions at 50 Hz; the inner loop runs at 500 Hz. We did not retune anything else.

The 0% vs 40–85% gap holds: vanilla and frame-stack stay at floor on the 6-DoF quadrotor, and FlyAda is the only policy with non-trivial transfer. FlyAda is, slightly

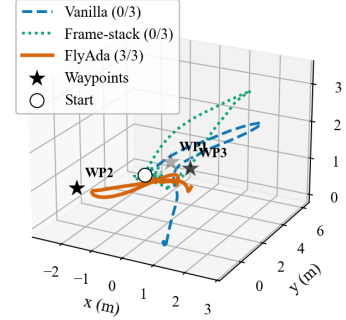


Fig. 4. MuJoCo Task B, hard condition (mass $\times$ 1.3, drag, wind 1 m/s), single seed.

counter-intuitively, better on the hard condition (0.85) than on nominal MuJoCo (0.40). A speculative reading, with a bit more mechanism attached: the 500 Hz attitude-rate PID and rotor dynamics sit underneath a policy re-planning only at 50 Hz ($K=4$), so they look like an added lag on top of whatever τ it trained against. f_ϕ only decodes velocity under the mass/drag/wind/delay axes of Sec. III; attitude/rotor lag enters ahead of the point-mass equation and is not one of those axes, so nominal MuJoCo asks the observer to extrapolate somewhere new, while the hard condition’s mass/drag/wind shift is exactly what f_ϕ was built for. Untested; confirming it needs an inner-loop-bandwidth sweep we did not run.

There is a real transfer gap nominally (40% vs 100% on the home sim), which is probably the cost of the unmodelled inner-loop dynamics. Re-training on 6-DoF data would likely close most of it; we did not do that.

XI. DISCUSSION

A few caveats and pieces of context.

Why not a recurrent policy? Recurrent encoders [26], [15], [16] are the standard partial-observation tool and would have been a reasonable choice. We ended up with the auxiliary observer for two reasons. The latent appends cleanly to the existing diffusion conditioning with no architectural change to the denoiser (one 73K-param MLP). And because the latent is explicit, the same-weights ablation (Sec. VII) and the linear probe (Sec. VIII) are straightforward design toggles rather than probes on an opaque hidden state. We did not run a head-to-head recurrent baseline; that is a fair follow-up.

Two practical limits. First, $\mathcal{L}_{\text{vel}}$ (Eq. 2) needs true velocity as a training-time target — mild for velocity (free to log even when withheld at deployment), but a real limitation and the whole mechanism: it does not obviously extend to quantities hard to instrument even in simulation, like drag or mass, where hiding a *parameter* rather than a *state channel* needs a different supervisory target (Sec. VIII). Second, everything above is sim-to-sim: a physical deployment needs the denoiser’s $T_{\text{sample}}=20$ DDIM steps to clear the 80 ms re-plan window ($K=4$ at 50 Hz) on embedded compute, and would expose f_ϕ ’s raw-position input to unmodelled sensor noise.

What the empirical finding says. The partial-observation failure we hit is about the objective rather than the input. A 3-frame stack supplies the same temporal information frame-by-frame; the diffusion MSE objective never asks the model to extract velocity from position deltas, and the policy still fails (1.1%). Adding the velocity-decoding head as an auxiliary loss puts that gradient back, the latent collapses onto a representation that recovers velocity, and the gap closes. The three components individually — an online-updated latent [2], an EMA-style update structurally similar to online longitudinal calibration in autonomous driving [3], and an auxiliary supervised head on a learned representation — are textbook; the empirical claim is about how they need to combine to close this particular failure.

Identifiability angle. With the auxiliary velocity-decoding signal, $\mathbf{z}_t$ is constrained to representations that decode to the true velocity through a linear map; the $R^2 = 0.992$ in Sec. VIII says the trained latent sits in this class. Without the auxiliary signal, the imitation MSE alone admits many feature spaces (Sec. V) and cannot prefer one. Auxiliary information recovering identifiability is the central claim in iVAE-style nonlinear ICA [28]; our results are broadly consistent, in the narrow setting of a single learned channel inside a diffusion policy’s conditioning vector. The latent-analysis numbers in Sec. VIII are consistent with the observer reading of the failure we report.

XII. CONCLUSION

Vanilla action-chunk diffusion holds up under the conventional mass/drag/wind/delay sweep when the full state is in the observation. Hiding velocity breaks it, and frame-stacking does not put it back. What did was an auxiliary-supervised observer with an EMA-updated latent, trained with privileged access to the true velocity that is itself never observed at deployment; a linear probe on that latent recovers the hidden velocity to $R^2 = 0.992$. Whether this generalises to higher-DoF dynamics, to other partial-observation regimes, or to parameters less directly observable than velocity, we did not test.

REFERENCES

- [1] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [2] L. Lyu, “AeroBridge-TTA: Test-time adaptive language-conditioned control for UAVs,” *arXiv preprint arXiv:2604.19059*, 2026.
- [3] S. Wang, C. Deng, and Q. Qi, “Efficient online calibration for autonomous vehicle’s longitudinal dynamical system: A Gaussian model approach,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] A. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *International Conference on Machine Learning (ICML)*, 2021.
- [6] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [7] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “RMA: Rapid motor adaptation for legged robots,” in *Robotics: Science and Systems (RSS)*, 2021.
- [8] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt, “Test-time training with self-supervision for generalization under distribution shifts,” in *International Conference on Machine Learning (ICML)*, 2020.
- [9] N. Hansen, R. Jangir, Y. Sun, G. Alenyà, P. Abbeel, A. A. Efros, L. Pinto, and X. Wang, “Self-supervised policy adaptation during deployment,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [10] M. O’Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural-fly enables rapid learning for agile flight in strong winds,” *Science Robotics*, vol. 7, no. 66, 2022.
- [11] G. Shi, X. Shi, M. O’Connell, R. Yu, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural lander: Stable drone landing control using learned dynamics,” in *2019 International Conference on Robotics and Automation (ICRA)*, 2019.
- [12] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Prentice Hall, 1991.
- [13] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning quadrupedal locomotion over challenging terrain,” *Science Robotics*, vol. 5, no. 47, 2020.
- [14] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning robust perceptive locomotion for quadrupedal robots in the wild,” *Science Robotics*, vol. 7, no. 62, 2022.
- [15] M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, et al., “Learning dexterous in-hand manipulation,” *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.
- [16] T. Ni, B. Eysenbach, and R. Salakhutdinov, “Recurrent model-free RL can be a strong baseline for many POMDPs,” in *International Conference on Machine Learning (ICML)*, 2022.
- [17] S. Wang and K. Hauser, “Real-time stabilization of a falling humanoid robot using hand contact: An optimal control approach,” in *2017 IEEE-RAS 17th International Conference on Humanoid Robots (Humanoids)*. IEEE, 2017, pp. 454–461.
- [18] S. Wang and K. Hauser, “Realization of a real-time optimal control strategy to stabilize a falling humanoid robot with hand contact,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 4466–4473.
- [19] S. Wang and K. Hauser, “Unified multi-contact fall mitigation planning for humanoids via contact transition tree optimization,” in *2018 IEEE-RAS 18th International Conference on Humanoid Robots (Humanoids)*. IEEE, 2018, pp. 1–9.
- [20] A. M. Sharma, S. Wang, Y. M. Zhou, and A. Ruina, “Towards a maximally-robust self-balancing bicycle without reaction-moment gyroscopes or reaction wheels,” in *Bicycle and Motorcycle Dynamics 2016*, 2016.
- [21] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.
- [22] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [23] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [25] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [26] M. Hausknecht and P. Stone, “Deep recurrent Q-learning for partially observable MDPs,” *arXiv preprint arXiv:1507.06527*, 2015.
- [27] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2012, pp. 5026–5033.
- [28] I. Khenakhem, D. P. Kingma, R. P. Monti, and A. Hyvärinen, “Variational autoencoders and nonlinear ICA: A unifying framework,” in *AISTATS*, 2020.